

Тема 10. Встроенный модуль HTTP

Основы сетевого взаимодействия: request, response, status codes, headers, Content-Type

Цель занятия

- Сформировать понимание клиент-серверной архитектуры и научиться создавать базовый HTTP-сервер на Node.js без сторонних библиотек.
- Понять модель HTTP request → response.
- Разобрать структуру HTTP-сообщения: стартовая строка, заголовки, тело.
- Научиться читать метод, URL и заголовки входящего запроса.
- Научиться формировать статус, заголовки и тело ответа.
- Создать первый HTTP-сервер через встроенный модуль `node:http`.
- Понять роль Content-Type и `charset=utf-8`.
- Проверить сервер через браузер и `curl`.

Учебные вопросы

- 1. Как работает HTTP.
- 2. Статус-коды: язык сервера.
- 3. Первый сервер на Node.js через `node:http`.
- 4. Объект `req`: что клиент прислал серверу.
- 5. Объект `res`: как сервер формирует ответ.
- 6. `Content-Type` и кодировка UTF-8.



1. Как работает HTTP

Клиент, сервер, запрос и ответ

Что такое HTTP

- HTTP — протокол прикладного уровня, по которому клиент и сервер обмениваются сообщениями.
- Браузер отправляет HTTP-запрос к серверу.
- Сервер обрабатывает запрос и отправляет HTTP-ответ.
- Мобильные приложения и backend-сервисы тоже могут использовать HTTP.
- В этой лекции рассматривается базовая модель HTTP/1.1.
- Для первого Node.js-сервера достаточно встроенного модуля `node:http`.

Основной принцип: request → response

- Классическая HTTP-модель строится вокруг пары: клиентский запрос и серверный ответ.
- Клиент обычно инициирует обмен: браузер, curl, Postman или приложение отправляет запрос.
- Сервер ожидает запросы и отвечает на них.
- Один запрос должен получить один ответ.
- В рамках обычного HTTP сервер не начинает диалог сам без первоначального запроса клиента.
- Исключения вроде streaming и Server-Sent Events всё равно начинаются с запроса клиента.

Схема HTTP-обмена

TEXT

Client



HTTP Request

Server



HTTP Response

Client

Что важно

- Request — сообщение от клиента к серверу.
- Response — сообщение от сервера к клиенту.
- Node.js-программа будет играть роль сервера.
- Браузер или curl будет играть роль клиента.

Структура HTTP-сообщения

- В HTTP/1.1 сообщение можно условно разделить на стартовую строку, заголовки и тело.
- Стартовая строка кратко описывает запрос или ответ.
- Заголовки передают технические параметры сообщения.
- Пустая строка отделяет заголовки от тела.
- Тело содержит полезные данные: HTML, JSON, текст, изображение или другой контент.
- Тело может быть не только текстовым: например, изображение передаётся как бинарные данные.

Части HTTP-сообщения

Часть	В запросе	В ответе
Start line	GET /about HTTP/1.1	HTTP/1.1 200 OK
Headers	Host, User-Agent, Accept	Content-Type, Content-Length
Blank line	Отделяет headers от body	Отделяет headers от body
Body	Данные формы, JSON	HTML, JSON, текст, файл

Пример HTTP-запроса

HTTP

```
GET /menu HTTP/1.1  
Host: localhost:3000  
User-Agent: curl/8.0  
Accept: */*
```

Что важно

- GET — метод запроса.
- /menu — путь, который запрашивает клиент.
- HTTP/1.1 — версия протокола.
- Host и User-Agent — заголовки запроса.

Пример HTTP-ответа

HTTP

```
HTTP/1.1 200 OK  
Content-Type: text/html; charset=utf-8
```

```
<h1>Меню</h1>
```

Что важно

- 200 OK — статус успешного ответа.
- Content-Type сообщает тип тела ответа.
- Пустая строка отделяет заголовки от body.
- `<h1>Меню</h1>` — тело ответа.

HTTP и состояние

- HTTP сам по себе stateless: каждый запрос обрабатывается как отдельное сообщение.
- Протокол не заставляет сервер помнить клиента между запросами.
- Если нужно состояние, используют cookies, sessions, JWT или данные в базе.
- Один и тот же клиент может выполнить много отдельных HTTP-запросов.
- Каждый запрос содержит всю информацию, необходимую для его обработки или идентификации.

Важное уточнение: HTTP не всегда только текст

- В учебных примерах HTTP/1.1 удобно показывать как текст: стартовая строка и headers читаются человеком.
- Тело сообщения может быть текстовым или бинарным.
- HTML и JSON — текстовые форматы.
- JPEG, PNG, PDF и видео — бинарные данные.
- HTTP/2 и HTTP/3 используют бинарное фреймирование, но базовая логика request → response сохраняется.

curl -v

BASH

```
curl -v http://example.com
```

Что важно

- -v включает подробный режим.
- Строки со знаком > показывают запрос клиента.
- Строки со знаком < показывают ответ сервера.
- Так удобно увидеть стартовую строку и заголовки.

Что показывает curl -v

- > GET / HTTP/1.1 — стартовая строка запроса.
- > Host: example.com — заголовок запроса.
- < HTTP/1.1 200 OK — стартовая строка ответа.
- < Content-Type: text/html — заголовок ответа.
- После пустой строки начинается тело ответа.

Что показывает curl -v

```
C:\Users\user>curl -v http://google.com
* Trying 142.251.14.100:80...
* Connected to google.com (142.251.14.100) port 80 (#0)
> GET / HTTP/1.1          <-- СТАРТОВАЯ СТРОКА ЗАПРОСА
> Host: google.com        <-- ЗАГОЛОВОК (куда идем)
> User-Agent: curl/7.83.1  <-- ЗАГОЛОВОК (кто мы)
> Accept: */*
>
* Mark bundle as not supporting multiuse

* Mark bundle as not supporting multiuse
< HTTP/1.1 301 Moved Permanently    <-- СТАРТОВАЯ СТРОКА ОТВЕТА (Статус-код!)
< Location: http://www.google.com/   <-- ЗАГОЛОВОК (куда переехали)
< Content-Type: text/html; charset=UTF-8 <-- ЗАГОЛОВОК (что прислали)
< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-
e-inline' https: http;;report-uri https://csp.withgoogle.com/csp/gws/other-hp
< Date: Fri, 17 Apr 2026 12:21:59 GMT
< Expires: Sun, 17 May 2026 12:21:59 GMT
< Cache-Control: public, max-age=2592000

< X-XSS-Protection: 0
< X-Frame-Options: SAMEORIGIN
<
<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
* Connection #0 to host google.com left intact

C:\Users\user>
```



2. Статус-коды

Как сервер сообщает результат обработки

Зачем нужны статус-коды

- HTTP-статус показывает общий результат обработки запроса.
- Клиент быстро понимает, успешно ли обработан запрос.
- Браузер, frontend и API-клиент используют статус-код для логики обработки.
- Статус-код находится в стартовой строке HTTP-ответа.
- Тело ответа может дополнительно объяснять ошибку или содержать данные.

Группы статус-кодов

Группа	Смысл	Примеры
1xx	Информационные	100 Continue
2xx	Успех	200 OK, 201 Created
3xx	Перенаправление	301 Moved Permanently, 304 Not Modified
4xx	Ошибка клиента	400, 401, 403, 404
5xx	Ошибка сервера	500, 503

2xx — успешные ответы

- 200 OK — запрос успешно обработан.
- 201 Created — ресурс создан, например после POST-запроса.
- 204 No Content — успешно, но тела ответа нет.
- Для API важно выбирать статус по смыслу операции.

3xx — перенаправления

- 301 Moved Permanently — ресурс переехал на постоянный адрес.
- 302 Found — временное перенаправление.
- 304 Not Modified — клиент может использовать кэшированную версию.
- Фактическое поведение кэширования зависит от клиента и заголовков.

4xx — ошибки клиента

- 400 Bad Request — запрос некорректен или данные невалидны.
- 401 Unauthorized — пользователь не аутентифицирован; обычно нужен логин или токен.
- 403 Forbidden — пользователь известен, но доступ запрещён.
- 404 Not Found — ресурс по указанному URL не найден.
- 405 Method Not Allowed — метод не поддерживается для этого ресурса.

5xx — ошибки сервера

- 500 Internal Server Error — внутренняя ошибка приложения.
- 502 Bad Gateway — промежуточный сервер получил некорректный ответ.
- 503 Service Unavailable — сервис временно недоступен или перегружен.
- 5xx означает, что проблема на стороне сервера или серверной инфраструктуры.

Минимум для запоминания

1 200 — успешно.

2 201 — создано.

3 400 — плохой запрос.

4 401 — не аутентифицирован.

5 403 — доступ запрещён.

6 404 — не найдено.

7 500 — ошибка сервера.

8 Статус-код нужно задавать осознанно, иначе клиент может получить 200 даже при логической ошибке.

3. Первый сервер на Node.js

Встроенный модуль `node:http` и ES Modules

Почему node:http

- node:http — встроенный модуль Node.js для создания HTTP-серверов и клиентов.
- Для первого сервера не нужно устанавливать Express.
- Модуль входит в Node.js.
- Он показывает низкоуровневую механику HTTP.
- После node:http проще понять, что делает Express внутри.
- В современном ESM-проекте импортируем модуль через import.

Подготовка проекта

BASH

```
mkdir http-demo  
cd http-demo  
npm init -y
```

Что важно

- Создаём отдельную папку проекта.
- `npm init -y` создаёт `package.json`.
- Далее добавим `type: module`.
- Сервер будет находиться в файле `index.js`.

package.json для ES Modules

JSON

```
{
  "name": "http-demo",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node index.js",
    "dev": "node --watch index.js"
  }
}
```

Что важно

- "type": "module" включает ES Modules.
- npm start запускает сервер обычным способом.
- npm run dev запускает сервер в watch mode.
- node --watch перезапускает процесс при изменении файла.

Минимальный HTTP-сервер

JAVASCRIPT

```
import { createServer } from 'node:http';

const PORT = 3000;

const server = createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain; charset=utf-8');
  res.end('Hello! Это мой первый Node.js сервер');
});

server.listen(PORT, () => {
  console.log(`Сервер запущен: http://localhost:${PORT}`);
  console.log('Нажмите Ctrl+C, чтобы остановить сервер');
});
```

Что важно

- `createServer` создаёт HTTP-сервер.
- Функция `(req, res)` вызывается на каждый входящий запрос.
- `res.statusCode` задаёт статус ответа.
- `res.setHeader` задаёт Content-Type.
- `res.end` завершает ответ.

CommonJS и ES Modules для http

CommonJS

- `const http = require('http');`
- `http.createServer(...);`
- Старый стиль Node.js.
- В этой версии лекции заменён.

ES Modules

- `import { createServer } from 'node:http';`
- `createServer(...);`
- Современный стиль проекта.
- Используется в курсе далее.

localhost

- localhost — имя, которое обычно указывает на текущий компьютер.
- Запрос к localhost не уходит в Интернет.
- Он возвращается на эту же машину.
- IPv4-адрес loopback обычно 127.0.0.1.
- В IPv6 аналогичный loopback-адрес — ::1.
- Для проверки локального сервера используем `http://localhost:3000`.

Порт

- Порт — номер логического входа, на котором приложение принимает соединения.
- IP-адрес указывает на компьютер.
- Порт указывает на конкретную службу внутри компьютера.
- Один сервер может слушать 3000, другой 8080.
- HTTP по умолчанию использует 80.
- HTTPS по умолчанию использует 443.
- В учебных проектах часто используют 3000.

Почему сервер не закрывается сам

- Обычный скрипт выполнил команды и завершился.
- HTTP-сервер открывает порт и ждёт входящие подключения.
- Открытый сервер удерживает event loop активным.
- Процесс будет работать, пока сервер не остановить вручную.
- В терминале сервер останавливают через Ctrl+C.

Запуск и проверка

BASH

```
npm start
```

```
# во втором терминале:  
curl http://localhost:3000
```

Что важно

- npm start запускает сервер.
- curl отправляет HTTP-запрос.
- В ответе должна появиться строка Hello.
- Можно также открыть `http://localhost:3000` в браузере.



4. Объект req

Что клиент прислал серверу

req: IncomingMessage

- req — объект входящего HTTP-запроса. Он содержит информацию о том, что клиент хочет получить или отправить.
- req.url содержит путь и query string.
- req.method содержит HTTP-метод.
- req.headers содержит заголовки запроса.
- req обычно читают, а не изменяют.
- Для тела POST-запроса нужны отдельные обработчики data и end.

Основные свойства req

Свойство	Пример	Зачем нужно
req.url	/users?id=5	Определить путь и query
req.method	GET	Понять действие клиента
req.headers	{ host, user-agent }	Прочитать технические данные запроса
req.headers['user-agent']	Mozilla/5.0...	Узнать клиентское приложение

Логирование req

JAVASCRIPT

```
const server = createServer((req, res) => {  
  console.log('URL:', req.url);  
  console.log('Method:', req.method);  
  console.log('User-Agent:', req.headers['user-agent']);  
  
  res.setHeader('Content-Type', 'text/plain; charset=utf-8');  
  res.end('Запрос получен');  
});
```

Что важно

- req.url показывает путь запроса.
- req.method показывает метод.
- req.headers хранит заголовки в нижнем регистре.
- Ответ всё равно нужно завершить через res.end().

URL и query string

JAVASCRIPT

```
const url = new URL(req.url, `http://${req.headers.host}`);  
  
console.log(url.pathname); // /users  
console.log(url.searchParams.get('id')); // 5
```

Что важно

- req.url может содержать путь и query string.
- new URL помогает безопасно разобрать адрес.
- pathname — путь без query.
- searchParams позволяет получить параметры запроса.

req.method

- GET обычно используется для получения данных.
- POST обычно используется для отправки данных на сервер.
- PUT часто означает полную замену ресурса.
- PATCH часто означает частичное изменение ресурса.
- DELETE обычно используется для удаления.
- В одном и том же URL разные методы могут означать разные действия.



5. Объект res

Как сервер формирует ответ

res: ServerResponse

- `res` — объект ответа. Через него сервер задаёт статус, заголовки и тело ответа.
- `res.statusCode` задаёт статус ответа.
- `res.setHeader` задаёт один заголовок.
- `res.writeHead` задаёт статус и заголовки вместе.
- `res.write` отправляет часть тела ответа.
- `res.end` завершает ответ.
- Если не вызвать `res.end()`, клиент будет ждать завершения ответа.

Основные методы и свойства res

Инструмент	Назначение
<code>res.statusCode = 404</code>	Установить статус ответа
<code>res.setHeader(name, value)</code>	Добавить или изменить заголовок до отправки
<code>res.writeHead(status, headers)</code>	Отправить статус и заголовки
<code>res.write(chunk)</code>	Записать часть тела ответа
<code>res.end(data)</code>	Завершить ответ и при необходимости отправить последний data

Порядок формирования ответа

- Сначала задаём статус и заголовки.
- Затем отправляем тело ответа.
- В конце обязательно вызываем `res.end()`.
- После отправки заголовков их уже нельзя свободно менять.
- После `res.end()` нельзя снова писать в ответ.

Правильный порядок

JAVASCRIPT

```
res.statusCode = 200;  
res.setHeader('Content-Type', 'text/html; charset=utf-8');  
res.write('<h1>Главная страница</h1>');  
res.end();
```

Что важно

- Сначала статус.
- Затем Content-Type.
- Потом тело ответа.
- В конце завершение через end().

Ошибка: запись после end()

JAVASCRIPT

```
res.end('Ответ завершён');
```

```
// Так делать нельзя:  
res.write('Ещё данные');
```

Что важно

- `res.end()` завершает ответ.
- После завершения писать в ответ уже нельзя.
- Node.js может выдать ошибку `write after end`.
- Это частая ошибка начинающих.

HTML-ответ

JAVASCRIPT

```
res.writeHead(200, {  
  'Content-Type': 'text/html; charset=utf-8'  
});  
  
res.end('<h1>Привет!</h1><p>Это HTML-страница.</p>');
```

Что важно

- text/html говорит браузеру обработать HTML-теги.
- charset=utf-8 нужен для корректной кириллицы.
- writeHead задаёт статус и заголовки одной командой.
- end отправляет тело и завершает ответ.

JSON-ответ

JAVASCRIPT

```
const data = {  
  message: 'Hello API',  
  status: 'ok'  
};  
  
res.writeHead(200, {  
  'Content-Type': 'application/json; charset=utf-8'  
});  
  
res.end(JSON.stringify(data));
```

Что важно

- application/json сообщает клиенту, что пришёл JSON.
- JSON.stringify превращает объект JavaScript в JSON-строку.
- Такой формат используется в REST API.
- Это подготовка к будущему Express API.



6. Content-Type и UTF-8

Как клиент понимает формат ответа

Content-Type

- Content-Type — заголовок, который сообщает клиенту тип данных в теле сообщения.
- Без Content-Type клиент может попытаться угадать формат.
- Браузер по-разному обрабатывает text/plain и text/html.
- API-клиенты ожидают application/json для JSON-ответов.
- Для русского текста важно указывать charset=utf-8.

Важные MIME-типы

MIME-тип	Что означает
text/plain; charset=utf-8	Обычный текст
text/html; charset=utf-8	HTML-документ
application/json; charset=utf-8	JSON-данные
image/png	PNG-изображение
image/jpeg	JPEG-изображение

text/plain и text/html

text/plain

- Строка показывается как обычный текст.
- `<h1>Hello</h1>` будет виден как текст.
- HTML-теги не отрисовываются.
- Подходит для простых текстовых ответов.

text/html

- Браузер интерпретирует HTML.
- `<h1>Hello</h1>` станет заголовком.
- Создаётся DOM-структура.
- Подходит для веб-страниц.

Кириллица и UTF-8

JAVASCRIPT

```
res.writeHead(200, {  
  'Content-Type': 'text/html; charset=utf-8'  
});  
  
res.end('<h1>Привет, мир!</h1>');
```

Что важно

- `charset=utf-8` сообщает кодировку текста.
- Без кодировки русские буквы могут отображаться некорректно.
- UTF-8 — стандартный выбор для современных веб-приложений.
- Эту часть лучше указывать во всех текстовых ответах.



7. Практика

Мини-сервер с маршрутами

Практическая задача

- Создать HTTP-сервер с несколькими маршрутами: главная страница, страница about, JSON API и 404.
- GET / возвращает HTML главной страницы.
- GET /about возвращает HTML страницы о проекте.
- GET /api/status возвращает JSON.
- Все остальные адреса возвращают 404.
- Код пишем на ES Modules через node:http.

Сервер с маршрутами

```
import { createServer } from 'node:http';

const PORT = 3000;

const server = createServer((req, res) => {
  const url = new URL(req.url, `http://${req.headers.host}`);

  if (req.method === 'GET' && url.pathname === '/') {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    return res.end('<h1>Главная</h1>');
  }

  if (req.method === 'GET' && url.pathname === '/about') {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    return res.end('<h1>О проекте</h1>');
  }

  if (req.method === 'GET' && url.pathname === '/api/status') {
    res.writeHead(200, { 'Content-Type': 'application/json; charset=utf-8' });
    return res.end(JSON.stringify({ status: 'ok' }));
  }

  res.writeHead(404, { 'Content-Type': 'text/plain; charset=utf-8' });
  res.end('404 Not Found');
});

server.listen(PORT, () => {
  console.log(`http://localhost:${PORT}`);
});
```

Что важно

- new URL помогает отделить pathname от query string.
- return res.end(...) останавливает обработчик после ответа.
- Для HTML используем text/html.
- Для JSON используем application/json.
- Неизвестный путь получает 404.

Проверка через curl

BASH

```
curl -i http://localhost:3000/  
curl -i http://localhost:3000/about  
curl -i http://localhost:3000/api/status  
curl -i http://localhost:3000/unknown
```

Что важно

- -i показывает заголовки ответа.
- На / ожидаем 200 и HTML.
- На /api/status ожидаем 200 и JSON.
- На /unknown ожидаем 404.

Итоги лекции

1

HTTP строится вокруг пары request → response.

2

В HTTP/1.1 стартовая строка и заголовки читаемы как текст, но тело может быть бинарным.

3

Статус-код показывает результат обработки.

4

node:http позволяет создать сервер без Express.

5

В ES Modules используем `import { createServer } from 'node:http'`.

6

req содержит данные запроса: url, method, headers.

7

res формирует ответ: statusCode, headers, body, end().

8

Content-Type сообщает клиенту формат ответа.

9

charset=utf-8 нужен для корректной кириллицы.

Контрольные вопросы

- 1 Кто обычно инициирует обмен в HTTP: клиент или сервер?
- 2 Из каких частей состоит HTTP/1.1 сообщение?
- 3 Почему фраза «HTTP — это только текст» неточна?
- 4 Что означает статус 200?
- 5 Чем отличаются 401 и 403?
- 6 Что произойдёт, если при ошибке забыть изменить статус-код?
- 7 Зачем нужен порт 3000 в учебном сервере?
- 8 Что показывает `req.url`?
- 9 Что показывает `req.method`?
- 10 Зачем нужен `res.end()`?
- 11 Можно ли вызвать `res.write()` после `res.end()`?
- 12 Чем отличаются `Content-Type: text/plain` и `text/html`?
- 13 Почему для русского текста нужно `charset=utf-8`?
- 14 Какой `Content-Type` нужен для JSON-ответа?